

Below are complete file contents to implement an in-browser editable PDF editor that preserves the original PDF template and writes edits back into a copy of the original PDF using `pdfjs-dist` and `pdf-lib`.

How to use: 1. Replace or add the files below into your project's `src/` (and `public/`) matching the paths shown. 2. `npm install pdf-lib pdfjs-dist` (or `npm i pdf-lib pdfjs-dist`). 3. `npm run dev` and open the app. Upload a sample PDF and edit.

Files

package.json (dependencies section)

```
{
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "pdf-lib": "^1.17.1",
    "pdfjs-dist": "^3.5.141",
    "vite": "^4.0.0"
  }
}
```

src/utils/template_pdf_manager.js

```
// template_pdf_manager.js
// Responsibilities:
// - provide function to apply edits (from overlay) back into a copy of the
//   original PDF
// - fetch font files from public/fonts/ and embed into new PDF using pdf-lib

import { PDFDocument, rgb, degrees, StandardFonts } from 'pdf-lib'

// Load font bytes from a public path and return ArrayBuffer
async function fetchFontBytes(fontPath) {
  try {
    const resp = await fetch(fontPath);
    if (!resp.ok) throw new Error(`Failed to fetch font ${fontPath}`);
    return await resp.arrayBuffer();
  } catch (err) {
    console.warn('fetchFontBytes error', err);
    return null;
  }
}
```

```

// fontMap: mapping from detected pdf font name to public relative path
// e.g. { 'Calibri': '/fonts/Calibri.ttf', 'Times-Roman': '/fonts/
TimesNewRoman.ttf' }
export async function applyEditsToPdf(originalPdfBytes, edits, fontMap = {})
{
  // edits: array of { pageIndex, x, y, text, fontName, fontSize, color?,
rotate?, maxWidth? }
  const pdfDoc = await PDFDocument.load(originalPdfBytes);

  // Preload and embed fonts
  const embeddedFonts = {};
  for (const key of Object.keys(fontMap)) {
    const path = fontMap[key];
    const bytes = await fetchFontBytes(path);
    if (bytes) {
      try {
        embeddedFonts[key] = await pdfDoc.embedFont(bytes);
      } catch (e) {
        console.warn('embed font failed for', key, e);
      }
    }
  }

  // Always have a fallback standard font
  const helv = await pdfDoc.embedFont(StandardFonts.Helvetica);

  for (const edit of edits) {
    const {
      pageIndex,
      x, y,
      text,
      fontName,
      fontSize = 12,
      color = { r: 0, g: 0, b: 0 },
      rotate = 0,
      maxWidth = undefined
    } = edit;
    const pages = pdfDoc.getPages();
    if (!pages[pageIndex]) continue;
    const page = pages[pageIndex];

    const font = embeddedFonts[fontName] || helv;

    // pdf-lib coordinate space origin is bottom-left, same as pdfjs user
    space.
    const opts = {
      x,
      y,
      size: fontSize,
      font,
      color: rgb(color.r, color.g, color.b)
    }
  }
}

```

```

};
if (typeof maxWidth !== 'undefined') opts.maxWidth = maxWidth;
if (rotate) opts.rotate = degrees(rotate);

// If text contains newlines, draw each line stacked
if (text.includes('\n')) {
  const lines = text.split('\n');
  const lineHeight = fontSize * 1.04;
  for (let i = 0; i < lines.length; i++) {
    page.drawText(lines[i], { ...opts, y: y - i * lineHeight });
  }
} else {
  page.drawText(text, opts);
}
}

const out = await pdfDoc.save();
return out;
}

```

src/components/PDFEditor.jsx

```

// PDFEditor.jsx
// Top-level component that handles upload, parsing with pdfjs, extracted
text items, overlay editing

import React, { useState, useRef, useEffect } from 'react';
import { getTextItemsFromPdf } from '../utils/pdfjs_extractor';
import PDFPage from './PDFPage';
import { applyEditsToPdf } from '../utils/template_pdf_manager';

export default function PDFEditor() {
  const [pdfBytes, setPdfBytes] = useState(null);
  const [textItems, setTextItems] = useState([]); // array of per-page arrays
  const [pagesMeta, setPagesMeta] = useState([]); // { width, height }
  const fileRef = useRef(null);

  async function onFileChange(e) {
    const file = e.target.files[0];
    if (!file) return;
    const bytes = await file.arrayBuffer();
    setPdfBytes(bytes);

    // extract text items (per page)
    const { itemsByPage, pages } = await getTextItemsFromPdf(bytes);
    setTextItems(itemsByPage);
    setPagesMeta(pages.map(p => ({ width: p.width, height: p.height })));
  }
}

```

```

// edits stored as map pageIndex -> [{...item, editedText}]
function onUpdateItem(pageIndex, itemIndex, editedText) {
  setTextItems(prev => {
    const copy = prev.map(arr => arr.slice());
    const item = copy[pageIndex][itemIndex];
    copy[pageIndex][itemIndex] = { ...item, editedText };
    return copy;
  });
}

async function onExport() {
  if (!pdfBytes) return alert('Upload a PDF first');
  // gather edits
  const edits = [];
  for (let pi = 0; pi < textItems.length; pi++) {
    const pageArr = textItems[pi] || [];
    for (let ti = 0; ti < pageArr.length; ti++) {
      const it = pageArr[ti];
      const text = (it.editedText !== undefined) ? it.editedText : it.str;
      // only write if text differs
      if (text !== it.str) {
        edits.push({
          pageIndex: pi,
          x: it.x,
          y: it.y,
          text,
          fontName: it.fontName || 'Helvetica',
          fontSize: it.fontSize || 12,
          maxWidth: it.width
        });
      }
    }
  }
}

// fontMap: map detected font name -> public path (adjust to your fonts)
const fontMap = {
  'Calibri': '/fonts/Calibri.ttf',
  'Times-Roman': '/fonts/TimesNewRoman.ttf',
  'Helvetica': '/fonts/Helvetica.ttf'
};

const newPdfBytes = await applyEditsToPdf(pdfBytes, edits, fontMap);
const blob = new Blob([newPdfBytes], { type: 'application/pdf' });
const url = URL.createObjectURL(blob);
const a = document.createElement('a');
a.href = url;
a.download = 'edited.pdf';
a.click();
URL.revokeObjectURL(url);
}

```

```

return (
  <div style={{ padding: 12 }}>
    <h2>PDF Editor</h2>
    <input ref={fileRef} type="file" accept="application/pdf"
onChange={onFileChange} />
    <button onClick={onExport} disabled={!pdfBytes} style={{ marginLeft:
8 }}>Export edited PDF</button>

    <div style={{ marginTop: 12 }}>
      {textItems.map((pageArr, i) => (
        <div key={i} style={{ marginBottom: 32 }}>
          <h3>Page {i + 1}</h3>
          <PDFPage
            pageIndex={i}
            pdfBytes={pdfBytes}
            textItems={pageArr}
            pageMeta={pagesMeta[i]}
            onUpdateItem={(itemIndex, editedText) => onUpdateItem(i,
itemIndex, editedText)}
          />
        </div>
      ))}
    </div>
  </div>
);
}

```

src/components/PDFPage.jsx

```

// PDFPage.jsx
// Renders a page canvas using pdfjs and overlays contentEditable divs

import React, { useEffect, useRef, useState } from 'react';
import * as pdfjsLib from 'pdfjs-dist/legacy/build/pdf';
import 'pdfjs-dist/web/pdf_viewer.css';

// pdfjs worker
pdfjsLib.GlobalWorkerOptions.workerSrc = `https://unpkg.com/pdfjs-
dist@3.5.141/build/pdf.worker.min.js`;

export default function PDFPage({ pageIndex, pdfBytes, textItems = [],
pageMeta = {}, onUpdateItem }) {
  const canvasRef = useRef(null);
  const containerRef = useRef(null);
  const [scale, setScale] = useState(1);

  useEffect(() => {

```

```

if (!pdfBytes) return;
let mounted = true;
(async () => {
  const loadingTask = pdfjsLib.getDocument({ data: pdfBytes });
  const pdf = await loadingTask.promise;
  const page = await pdf.getPage(pageIndex + 1);
  const viewport = page.getViewport({ scale: 1 });
  const canvas = canvasRef.current;
  const ctx = canvas.getContext('2d');

  // fit to container width
  const containerWidth = containerRef.current.clientWidth || 800;
  const fitScale = containerWidth / viewport.width;
  setScale(fitScale);
  const scaledViewport = page.getViewport({ scale: fitScale });

  canvas.width = scaledViewport.width;
  canvas.height = scaledViewport.height;

  const renderContext = {
    canvasContext: ctx,
    viewport: scaledViewport
  };
  await page.render(renderContext).promise;
})();

return () => { mounted = false; };
}, [pdfBytes, pageIndex]);

const pageHeight = pageMeta?.height || 800;

function toDomPos(item) {
  // item: has x,y,width,height in PDF user space with origin bottom-left
  // convert to DOM top-left with scaling
  const s = scale;
  const top = (pageHeight - item.y - item.height) * s;
  return {
    left: item.x * s,
    top,
    width: item.width * s,
    height: item.height * s,
    fontSize: (item.fontSize || 12) * s
  };
}

return (
  <div ref={containerRef} style={{ position: 'relative', width: '100%',
  maxWidth: 900 }}>
    <canvas ref={canvasRef} style={{ width: '100%', display: 'block' }} />

    {/* Overlay editable divs */}

```

```

    {textItems.map((it, idx) => {
      const pos = toDomPos(it);
      return (
        <div
          key={idx}
          contentEditable
          suppressContentEditableWarning
          onBlur={e => onUpdateItem(idx, e.target.innerText)}
          style={{
            position: 'absolute',
            left: pos.left,
            top: pos.top,
            width: pos.width,
            minHeight: pos.height,
            fontSize: pos.fontSize,
            lineHeight: `${pos.fontSize}px`,
            fontFamily: it.fontName || 'inherit',
            whiteSpace: 'pre-wrap',
            overflow: 'hidden',
            background: 'transparent'
          }}
        >
          {it.editedText !== undefined ? it.editedText : it.str}
        </div>
      );
    })}
  </div>
);
}

```

src/utils/pdfjs_extractor.js

```

// pdfjs_extractor.js
// Minimal extractor that returns text items per page including bounding
boxes and font size

import * as pdfjsLib from 'pdfjs-dist/legacy/build/pdf';
pdfjsLib.GlobalWorkerOptions.workerSrc = `https://unpkg.com/pdfjs-
dist@3.5.141/build/pdf.worker.min.js`;

export async function getTextItemsFromPdf(pdfBytes) {
  const loadingTask = pdfjsLib.getDocument({ data: pdfBytes });
  const pdf = await loadingTask.promise;
  const numPages = pdf.numPages;
  const itemsByPage = [];
  const pages = [];

  for (let i = 1; i <= numPages; i++) {

```

```

const page = await pdf.getPage(i);
const viewport = page.getViewport({ scale: 1 });
const textContent = await page.getTextContent();

const items = textContent.items.map(it => {
  // transform: [a, b, c, d, e, f] from pdfjs
  // For each item, there is a transform we can use to compute x,y
  const tx = it.transform;
  // tx = [scaleX, skewX, skewY, scaleY, translateX, translateY]
  const x = tx[4];
  const y = tx[5];
  const fontSize = Math.hypot(tx[0], tx[1]);
  const width = it.width || (it.str.length * fontSize * 0.55);
  const height = fontSize;
  const fontName = (it.fontName || '').split('+').pop();

  return {
    str: it.str,
    x,
    y,
    width,
    height,
    fontSize,
    fontName
  };
});

itemsByPage.push(items);
pages.push({ width: viewport.width, height: viewport.height });
}

return { itemsByPage, pages };
}

```

public/fonts/

Place font files you own/are licensed to use into `public/fonts/` with filenames used in `PDFEditor.jsx` `fontMap` (e.g. `Calibri.ttf`, `TimesNewRoman.ttf`, `Helvetica.ttf`). If you can't provide those fonts, map fonts to free alternatives (Roboto, OpenSans) in the `fontMap`.

README / run steps

1. Install deps: `npm install`
2. Start dev server: `npm run dev`
3. Open the app and upload a PDF.
4. Click into text regions to edit. Click Export to download edited PDF.

Notes, limitations and next steps

- Font fidelity depends on embedding matching TTF/OTF files. If an exact font isn't available, visual differences can occur.
- This extractor is simplified and may not perfectly compute bounding boxes for complex PDFs. The repo you uploaded had a more advanced extractor — you can swap this simplified extractor for the repository's `extract_pdf_positions.js` if you prefer.
- Rotations, complex transforms, ligatures, and subset-embedded font metrics may require additional handling.

If you want, I can now: - Provide a patch replacing specific files in your zip. - Convert parts of this into ready-to-paste diffs for the exact file names in your project.

End of file list.